

SMART ERP / LOGIXONE

Manual de instalación y puesta en marcha de Smart ERP en Ubuntu

Producto: LogixOne / repositorio `smart-erp`

Edición: 1.0 - entorno local o servidor de demostración

Fecha de verificación documental: 12 de agosto de 2026

Plataforma objetivo: Ubuntu 24.04 LTS, arquitectura AMD64

Perfil funcional del ejemplo: `with-purchasing-demo`

Audiencia: persona nueva que administrará, demostrará o evaluará el sistema

Contenido

1. Cómo usar este manual
2. Términos que debe conocer
3. Arquitectura que se levantará
4. Etapa 1 - Diagnosticar el equipo Ubuntu
5. Etapa 2 - Instalar Docker Engine, Compose y Git
6. Etapa 3 - Descargar el proyecto desde GitLab
7. Etapa 4 - Crear secretos y configuración local
8. Etapa 5 - Construir las imágenes
9. Etapa 6 - Iniciar todos los servicios
10. Etapa 7 - Validar salud y abrir el navegador
11. Operación diaria: estado, logs, reinicio y parada
12. Actualizar a una nueva versión
13. Cambiar puertos o publicar detrás de un proxy
14. Diagnóstico y recuperación
15. Lista de comprobación rápida
16. Validación de fuentes y alcance de datos
17. Seguridad, límites y soporte
18. Referencias canónicas

1. Cómo usar este manual

Estado de soporte: este procedimiento levanta el entorno reproducible mediante Docker Compose. No es un instalador Linux ni una guía de producción. Keycloak se ejecuta en modo de desarrollo y los puertos quedan restringidos al propio equipo. Para producción hacen falta decisiones adicionales de TLS, proxy, DNS, secretos, respaldo, monitoreo y alta disponibilidad.

Siga las etapas en orden la primera vez. Una vez instalado, use la sección 15 como referencia diaria y la sección 14 para diagnosticar problemas. No copie únicamente el comando: lea también el resultado esperado y la recuperación segura de cada etapa.

Al terminar podrá:

1. comprobar que Ubuntu y Docker son aptos para el entorno de demostración;
2. descargar el código desde el GitLab principal;
3. crear secretos locales sin incorporarlos a Git;
4. construir las imágenes de aplicación y migraciones;
5. iniciar PostgreSQL, Keycloak, el migrador y la aplicación;
6. abrir LogixOne desde un navegador local o por un túnel SSH;
7. detener, reiniciar, actualizar y diagnosticar el entorno sin borrar datos.

Qué no cubre

- certificación productiva en Ubuntu;
- publicación directa en Internet;
- instalación de un proxy TLS o un nombre DNS;
- clúster, alta disponibilidad o recuperación ante desastres;
- aceptación funcional por una persona independiente;
- creación de un instalador Linux.

Convenciones de los comandos

- Ejecute los comandos desde una terminal Bash.
- El símbolo `$` representa el prompt y no se escribe.
- Los bloques se pueden copiar completos salvo que indiquen lo contrario.
- Los comandos del proyecto suponen que su usuario puede acceder al socket de Docker. Si no puede, anteponga `sudo` a cada comando `docker`.
- Sustituya los valores entre `<` y `>`; no escriba los signos angulares.
- La carpeta elegida en los ejemplos es `$HOME/proyectos/smart-erp`.

2. Términos que debe conocer

Término	Explicación para una persona nueva
Ubuntu	Sistema operativo Linux sobre el que se ejecutarán Docker y Git.
Terminal	Ventana donde se escriben comandos. En Ubuntu puede abrirse con <code>Ctrl+Alt+T</code> .
Repositorio	Carpeta versionada que contiene código, infraestructura y documentación del proyecto.
GitLab	Servidor principal del código de este proyecto. GitHub queda como remoto secundario.
Git	Herramienta que descarga versiones del repositorio y permite actualizarlas.
Rama <code>main</code>	Línea principal del código. Este manual parte de ella.
Docker Engine	Servicio que crea y ejecuta contenedores Linux aislados.
Imagen	Paquete inmutable con una aplicación y su runtime. Se construye antes de crear un contenedor.
Contenedor	Instancia en ejecución de una imagen. Puede recrearse sin borrar automáticamente los volúmenes.
Docker Compose	Herramienta que inicia varios servicios coordinados a partir de <code>compose.yaml</code> .
Servicio	Unidad declarada en Compose. Este entorno tiene <code>postgres</code> , <code>migrator</code> , <code>keycloak</code> y <code>app</code> .
Volumen	Almacenamiento persistente administrado por Docker. Conserva PostgreSQL y Keycloak aunque los contenedores se creen.
Secreto	Contraseña o clave sensible. Aquí se guarda en archivos locales bajo <code>.tools/secrets/</code> , nunca en Git.
PostgreSQL	Base de datos relacional del ERP.
Migración	Cambio versionado y auditable que crea o evoluciona tablas, funciones y restricciones.
Migrador	Contenedor de una sola ejecución que aplica migraciones antes de arrancar la aplicación.
Keycloak	Servicio que autentica usuarios y emite la identidad utilizada por LogixOne.
OIDC	Protocolo con el que la aplicación delega el inicio de sesión en Keycloak.
WildFly	Servidor Jakarta EE incluido dentro de la imagen de la aplicación. No se instala globalmente en Ubuntu.
Health check	Comprobación automática del estado de un servicio. <code>live</code> indica que está vivo y <code>ready</code> que puede atender tráfico.

Término	Explicación para una persona nueva
Loopback	Dirección accesible sólo desde el mismo equipo: <code>127.0.0.1</code> . Es la publicación predeterminada.
Puerto	Número que identifica un servicio de red. Este manual usa <code>8080</code> para LogixOne y <code>8180</code> para Keycloak.
Túnel SSH	Conexión cifrada que hace que los puertos remotos aparezcan como puertos locales, sin publicarlos en Internet.
Perfil Maven	Selección de módulos que entran en la distribución. El ejemplo incluye Compras mediante <code>with-purchasing-demo</code> .
Digest	Huella criptográfica de una imagen. En ambientes compartidos permite ejecutar exactamente el artefacto aprobado.

3. Arquitectura que se levantará

El navegador nunca se conecta directamente a PostgreSQL. Compose crea dos redes internas para base de datos e identidad, y una red de borde para los puertos publicados. El migrador debe terminar correctamente antes de que arranque la aplicación.



Datos y objetos persistentes

Objeto	Propietario	Qué contiene	¿Se conserva con <code>docker compose down</code> ?
Código del repositorio	Git	Fuentes, Compose, Dockerfiles y manuales	Sí
<code>.tools/secrets/*.txt</code>	Operador local	Contraseñas y secreto OIDC	Sí; no está versionado
<code>compose.env.local</code>	Operador local	Configuración no sensible del entorno	Sí; no debe contener contraseñas
Imagen <code>logixone/app</code>	Docker	WAR, WildFly y módulos seleccionados	Sí, hasta una limpieza explícita de imágenes
Imagen <code>logixone/migrator</code>	Docker	Migraciones del kernel y plugins	Sí, hasta una limpieza explícita de imágenes
Volumen <code>postgres-data</code>	Compose/PostgreSQL	Datos y esquema del ERP	Sí
Volumen <code>keycloak-data</code>	Compose/Keycloak	Realm, usuarios y estado de identidad	Sí

Advertencia de pérdida de datos: `docker compose down` conserva los volúmenes. `docker compose down --volumes` los elimina. No use `--volumes` como solución de diagnóstico ni sin respaldo y autorización explícita.

4. Etapa 1 - Diagnosticar el equipo Ubuntu

Objetivo: comprobar sistema, arquitectura, memoria, disco, puertos y acceso de red antes de instalar.

Datos que lee: versión del sistema, arquitectura, recursos y sockets.

Datos que modifica: ninguno.

Tablas PostgreSQL afectadas: ninguna; la base todavía no existe.

Bosquejo orientativo de la terminal

```
+-----+
| usuario@ubuntu:~$ |
| Ubuntu 24.04 LTS   |
| Arquitectura: x86_64 |
| Memoria y disco: disponibles |
| Puertos 8080/8180: sin proceso escuchando |
+-----+
```

Diagrama de objetos afectados

```
[ Ubuntu /proc y /etc ] -- lectura --> [ Terminal ]
[ Tabla de sockets ]    -- lectura --> [ Terminal ]
[ PostgreSQL ]         -- sin acceso --> [ ninguna tabla ]
```

Comandos

```
cat /etc/os-release
uname -m
nproc
free -h
df -h /
ss -ltn | grep -E ':(8080|8180)\b' || true
getent hosts gitlab.cosmesoft.com.py
```

Resultado esperado

- `VERSION_ID` debe indicar una versión Ubuntu soportada por Docker; este manual apunta a `24.04`.
- `uname -m` debe devolver `x86_64`, porque las imágenes fijan `linux/amd64`.
- Los puertos `8080` y `8180` deberían estar libres.
- El nombre de GitLab debe resolver a una dirección IP.

El proyecto todavía no tiene una matriz Ubuntu oficialmente aprobada. Para una demostración completa reserve recursos suficientes para cuatro servicios y para construir Java dentro de Docker. Como referencia conservadora de laboratorio, use al menos 8 GiB de RAM y 30 GiB libres; para uso compartido mida la carga y apruebe un dimensionamiento específico.

Si algo falla

Síntoma	Significado	Recuperación segura
<code>aarch64</code> o <code>arm64</code>	La CPU no coincide con <code>linux/amd64</code>	Use una máquina AMD64. No quite la plataforma del Compose sin una decisión arquitectónica.
Puerto ocupado	Otro proceso escucha en el puerto	Identifique el proceso con <code>sudo ss -ttnp</code> . Cambie puertos sólo siguiendo la sección 13.
GitLab no resuelve	Falla DNS o red corporativa	Verifique VPN, proxy y DNS con el administrador; no inserte credenciales en comandos de diagnóstico.
Poco disco	El build puede quedar incompleto	Libere espacio de forma controlada. No borre volúmenes del proyecto.

5. Etapa 2 - Instalar Docker Engine, Compose y Git

Objetivo: instalar los únicos prerequisites globales del recorrido oficial. Java, Maven, WildFly y PostgreSQL se ejecutan dentro de imágenes o herramientas gobernadas por el proyecto.

Datos que modifica: paquetes APT, clave y repositorio oficial de Docker, servicio `docker`.

Tablas PostgreSQL afectadas: ninguna.

Bosquejo orientativo de la terminal

```
+-----+
| usuario@ubuntu:~$ sudo docker version |
| Client: Docker Engine ...             |
| Server: Docker Engine ...             |
| usuario@ubuntu:~$ sudo docker compose version |
| Docker Compose version ...            |
+-----+
```

Diagrama de objetos afectados

```
[ download.docker.com ] --> [ APT ] --> [ Docker Engine ]
[ archive.ubuntu.com ] --> [ APT ] --> [ Git/OpenSSL ]
[ PostgreSQL ]         --> sin acceso --> [ ninguna tabla ]
```

Instalar paquetes base

```
sudo apt update
sudo apt install -y ca-certificates curl git openssl
```

Agregar el repositorio oficial de Docker

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Cree `/etc/apt/sources.list.d/docker.sources` con este contenido. En una terminal interactiva puede copiar el bloque completo:

```
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Instale y habilite el motor:

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
```

Validar la instalación

```
sudo docker run --rm hello-world
sudo docker version
sudo docker compose version
sudo docker buildx version
git --version
```

El ejemplo `hello-world` debe terminar con un mensaje de instalación correcta. `docker version` debe mostrar tanto `Client` como `Server`.

Seguridad: pertenecer al grupo `docker` concede privilegios equivalentes a root. Este manual conserva `sudo` como opción segura y explícita. Si la política de su organización autoriza acceso directo al socket, siga la guía oficial de posinstalación y vuelva a iniciar sesión.

Firewall: Docker advierte que los puertos publicados pueden eludir reglas administradas sólo con UFW. Este proyecto limita los puertos a `127.0.0.1` de manera predeterminada; no cambie ese bind para “hacerlo visible” en la red.

6. Etapa 3 - Descargar el proyecto desde GitLab

Objetivo: obtener una copia de trabajo de la rama principal sin colocar tokens en la URL.

Datos que modifica: crea `$HOME/proyectos/smart-erp`.

Tablas PostgreSQL afectadas: ninguna.

Bosquejo orientativo de la terminal

```
+-----+
| usuario@ubuntu:~/proyectos$ git clone ... |
| Cloning into 'smart-erp'...               |
| usuario@ubuntu:~/proyectos/smart-erp$ git status |
| On branch main / working tree clean         |
+-----+
```

Diagrama de objetos afectados

```
[ GitLab origin/main ] -- lectura Git --> [ smart-erp/.git ]
|
|                                     +--> [ fuentes locales ]
[ PostgreSQL ]      -- sin acceso -----> [ ninguna tabla ]
```

Comandos

```
mkdir -p "$HOME/proyectos"
cd "$HOME/proyectos"
git clone https://gitlab.cosmesoft.com.py/cosmesoft/smart-erp.git
cd smart-erp
git switch main
git pull --ff-only origin main
git status --short --branch
git remote -v
```

Resultado esperado

- la rama actual es `main`;
- el árbol de trabajo está limpio;
- `origin` apunta a GitLab;
- si existe el remoto `github`, es secundario y no interviene en el arranque.

Si Git solicita autenticación, use el mecanismo aprobado por su organización, por ejemplo un administrador de credenciales o una clave SSH. No escriba un token dentro de la URL, de un script o de este manual.

7. Etapa 4 - Crear secretos y configuración local

Objetivo: crear cuatro secretos independientes y una configuración Compose local.

Datos que modifica: archivos locales bajo `.tools/secrets/` e `infra/compose/compose.env.local`.

Tablas PostgreSQL afectadas: ninguna todavía.

Qué representa cada dato

Archivo	Consumidor	Finalidad
postgres-password.txt	PostgreSQL, migrador y aplicación	Contraseña del usuario de base logixone
keycloak-admin-password.txt	Keycloak	Contraseña del administrador técnico local
oidc-client-secret.txt	Keycloak y aplicación	Secreto compartido del cliente OIDC logixone-web
demo-user-password.txt	Importación de Keycloak	Contraseña común de los usuarios ficticios de demostración

Bosquejo orientativo de la terminal

```
+-----+
| smart-erp/.tools/secrets/ |
| postgres-password.txt      | permiso 600 |
| keycloak-admin-password.txt | permiso 600 |
| oidc-client-secret.txt     | permiso 600 |
| demo-user-password.txt     | permiso 600 |
| infra/compose/compose.env.local |
+-----+
```

Diagrama de objetos afectados

```
[ OpenSSL aleatorio ] --> C [ .tools/secrets/*.txt ]
[ compose.env.example ] --> C/U [ compose.env.local ]
[ Git ] -- ignora secretos/configuración local
[ PostgreSQL ] -- sin acceso --> [ ninguna tabla ]
```

C significa crear y U actualizar. Ninguna operación SQL ocurre en esta etapa.

Crear secretos

```
cd "$HOME/proyectos/smart-erp"
install -d -m 700 .tools/secrets
umask 077
openssl rand -base64 32 > .tools/secrets/postgres-password.txt
openssl rand -base64 32 > .tools/secrets/keycloak-admin-password.txt
openssl rand -base64 32 > .tools/secrets/oidc-client-secret.txt
openssl rand -base64 24 > .tools/secrets/demo-user-password.txt
chmod 600 .tools/secrets/*.txt
```

Cada archivo debe contener una sola línea no vacía. No muestre su contenido en capturas, tickets, historial de terminal o mensajes.

Crear la configuración no sensible

```
cp infra/compose/compose.env.example infra/compose/compose.env.local
```

Abra el archivo con su editor y cambie al menos las dos imágenes:

```
LOGIXONE_APP_IMAGE=logixone/app:j11-s9-06-purchasing-demo-r5
LOGIXONE_MIGRATOR_IMAGE=logixone/migrator:j11-s9-06-purchasing-demo
```

Para una demostración local con empresas ficticias, puede cambiar temporalmente:

```
LOGIXONE_DEMO_PROVISIONING_ENABLED=true
```

Ese indicador autoriza el aprovisionamiento técnico de la demo al arrancar. Después del primer arranque correcto, vuelva a `false` y recree sólo la aplicación. No lo use como mecanismo general de alta de empresas ni en producción.

Validar rutas y Compose

```
test -s .tools/secrets/postgres-password.txt
test -s .tools/secrets/keycloak-admin-password.txt
test -s .tools/secrets/oidc-client-secret.txt
test -s .tools/secrets/demo-user-password.txt
sudo docker compose \
  --env-file infra/compose/compose.env.local \
  -f infra/compose/compose.yaml config --quiet
```

El último comando no debe producir salida ni errores. Si informa que falta un secreto, compruebe que ejecutó el comando desde la raíz del repositorio y que no cambió las rutas relativas del archivo local.

8. Etapa 5 - Construir las imágenes

Objetivo: compilar el baseline seleccionado y producir la imagen de aplicación y la imagen de migraciones.

Datos que modifica: caché de build e imágenes locales de Docker.

Tablas PostgreSQL afectadas: ninguna; construir no inicia la base.

Bosquejo orientativo de la terminal

```
+-----+
| smart-erp$ sudo docker build ... Dockerfile |
| => Maven verify / WAR / WildFly            |
| => naming to logixone/app:j11-s9-06-purchasing-demo-r5 |
| smart-erp$ sudo docker image ls logixone/*    |
+-----+
```

Diagrama de objetos afectados

```
[ código + pom.xml + mvnw ] -- R --> [ Docker BuildKit ]
                                   | C
                                   +--> [ logixone/app:tag ]
                                   +--> [ logixone/migrator:tag ]
[ PostgreSQL ] -- sin acceso -----> [ ninguna tabla ]
```

Construir la aplicación

```
sudo docker build --platform linux/amd64 \
--build-arg LOGIXONE_BUILD_MODE=verified \
--build-arg LOGIXONE_MAVEN_PROFILE=with-purchasing-demo \
--tag logixone/app:j11-s9-06-purchasing-demo-r5 \
--file infra/docker/Dockerfile .
```

Construir el migrador

```
sudo docker build --platform linux/amd64 \
--build-arg LOGIXONE_BUILD_MODE=verified \
--build-arg LOGIXONE_MAVEN_PROFILE=with-purchasing-demo \
--tag logixone/migrator:j11-s9-06-purchasing-demo \
--file infra/docker/Dockerfile.migrator .
```

`LOGIXONE_BUILD_MODE=verified` exige el corte verificado del build. `LOGIXONE_MAVEN_PROFILE` define qué plugins entran físicamente. Aplicación y migrador deben construirse con el mismo perfil para evitar desalinear código y esquema.

Validar las imágenes

```
sudo docker image inspect logixone/app:j11-s9-06-purchasing-demo-r5 \
--format '{{.Id}} {{.Os}}/{{.Architecture}}'
sudo docker image inspect logixone/migrator:j11-s9-06-purchasing-demo \
--format '{{.Id}} {{.Os}}/{{.Architecture}}'
```

Ambas deben indicar `linux/amd64`. Si el build falla, no continúe con una imagen anterior que tenga el mismo nombre. Conserve el log, corrija la causa y repita el build.

9. Etapa 6 - Iniciar todos los servicios

Objetivo: crear redes y volúmenes, iniciar PostgreSQL y Keycloak, aplicar migraciones y finalmente arrancar LogixOne.

Datos que modifica: contenedores, redes, volúmenes, esquema PostgreSQL y datos técnicos de Keycloak.

Tablas afectadas: tablas de historia Flyway y las tablas versionadas de `core` y de los plugins incluidos. No se usa actualización automática de Hibernate.

Bosquejo orientativo de la terminal

NAME	SERVICE	STATUS
logixone-postgres-1	postgres	Up (healthy)
logixone-migrator-1	migrator	Exited (0)
logixone-keycloak-1	keycloak	Up (healthy)
logixone-app-1	app	Up (healthy)

Diagrama de tablas y objetos afectados

```
[ migrator ] -- C/U DDL versionado --> [ core.flyway_schema_history ]
|                                     [ plg_*.flyway_schema_history ]
+-- C/U DDL versionado --> [ tablas, funciones y triggers de cada dueño ]
[ app ] ----- R/W por casos de uso --> [ core y esquemas privados de plugins ]
[ keycloak ] -- R/W -----> [ volumen keycloak-data ]
```

No hay relaciones JPA ni escrituras directas entre tablas privadas de plugins.

Iniciar y esperar salud

```
cd "$HOME/proyectos/smart-erp"
sudo docker compose \
  --env-file infra/compose/compose.env.local \
  -f infra/compose/compose.yaml \
  up -d --wait --wait-timeout 240
```

Comprobar el estado

```
sudo docker compose \
  --env-file infra/compose/compose.env.local \
  -f infra/compose/compose.yaml ps -a
```

El estado correcto es:

- `postgres` : Up y healthy ;
- `migrator` : Exited (0) , porque es una tarea de una sola ejecución;
- `keycloak` : Up y healthy ;

- `app` : `Up` y `healthy` .

No confunda terminado con error: `migrator` no debe quedar `Up` . `Exited (0)` significa que aplicó o validó todas las migraciones y terminó bien. Cualquier otro código bloquea el arranque de `app` .

Si habilitó la demo temporal

Cambie `LOGIXONE_DEMO_PROVISIONING_ENABLED=false` y recree sólo la aplicación:

```
sudo docker compose \  
--env-file infra/compose/compose.env.local \  
-f infra/compose/compose.yaml \  
up -d --no-deps --force-recreate app
```

Espere que vuelva a estar saludable antes de abrir el navegador.

10. Etapa 7 - Validar salud y abrir el navegador

Objetivo: confirmar que la aplicación puede atender tráfico y completar el inicio de sesión OIDC.

Datos que lee: endpoints de salud, configuración OIDC y datos autorizados del usuario.

Datos que modifica: sesión de navegador; el login actualiza estado técnico de Keycloak.

Tablas PostgreSQL afectadas: los health checks leen metadatos de disponibilidad; el recorrido funcional posterior usa las tablas del módulo y de seguridad según la acción.

Bosquejo orientativo del navegador

```
+-----+
| http://localhost:8080/logixone/ |
+-----+
| LogixOne |
| |
| [ Iniciar sesión ] |
| -> Keycloak / realm logixone |
| -> usuario demo.empresa.a |
| -> vuelve al panel de la empresa |
+-----+
```

Diagrama de datos afectados

```
[ navegador ] -- GET --> [ /health/live ] -- R --> [ vida del proceso ]
[ navegador ] -- GET --> [ /health/ready ] -- R --> [ DB + OIDC + plugins ]
[ navegador ] -- OIDC --> [ Keycloak realm logixone ]
[ app ] -- autorización --> [ core: identidad, empresa, roles, activaciones ]
```

Validar desde la terminal

```
curl -fsS http://127.0.0.1:8080/logixone/health/live
curl -fsS http://127.0.0.1:8080/logixone/health/ready
```

Ambos comandos deben terminar con código HTTP 200. Después abra:

```
http://localhost:8080/logixone/
```

Keycloak usa el realm `logixone` . Los usuarios ficticios incluidos son:

Usuario	Propósito
demo.sin.empresa	Demostrar un usuario autenticado sin empresa habilitada
demo.empresa.a	Demostrar un usuario con una empresa
demo.empresas.ab	Demostrar selección entre más de una empresa

La contraseña de los tres es el valor guardado en `.tools/secrets/demo-user-password.txt` . Consúltelo sólo en una terminal privada y no lo copie a documentación ni tickets.

Acceder a un servidor Ubuntu remoto

No cambie el bind a `0.0.0.0` . Desde su computadora, cree un túnel SSH:

```
ssh -L 8080:127.0.0.1:8080 \  
-L 8180:127.0.0.1:8180 \  
<usuario>@<servidor-ubuntu>
```

Mantenga esa sesión abierta y use en el navegador de su computadora:

```
http://localhost:8080/logixone/
```

El túnel transporta también `8180` , necesario para que la redirección a `keycloak.localhost` funcione desde el navegador local.

11. Operación diaria: estado, logs, reinicio y parada

Ver estado

```
sudo docker compose \
--env-file infra/compose/compose.env.local \
-f infra/compose/compose.yaml ps -a
```

Ver logs sin revelar secretos

```
sudo docker compose \
--env-file infra/compose/compose.env.local \
-f infra/compose/compose.yaml logs --no-color --tail=200 migrator

sudo docker compose \
--env-file infra/compose/compose.env.local \
-f infra/compose/compose.yaml logs --no-color --tail=200 postgres keycloak app
```

Antes de compartir un log, revise que no contenga tokens, contraseñas, datos personales o información real de una empresa.

Reiniciar sin recrear datos

```
sudo docker compose \
--env-file infra/compose/compose.env.local \
-f infra/compose/compose.yaml restart app keycloak
```

Detener preservando datos

```
sudo docker compose \
--env-file infra/compose/compose.env.local \
-f infra/compose/compose.yaml down
```

Volver a iniciar

```
sudo docker compose \
--env-file infra/compose/compose.env.local \
-f infra/compose/compose.yaml \
up -d --wait --wait-timeout 240
```

Objetos y tablas afectados

```
[ down ] --> elimina contenedores y redes del proyecto
          --> conserva [ postgres-data ] y [ keycloak-data ]
[ up ]    --> recrea contenedores y redes
          --> reutiliza volúmenes y valida/aplica migraciones inmutables
[ restart ] --> reinicia procesos; no elimina tablas ni volúmenes
```

12. Actualizar a una nueva versión

Una actualización puede incluir migraciones. Haga primero un respaldo, lea la evidencia de la iteración y utilice etiquetas nuevas o digests aprobados.

Secuencia segura

1. Confirme que no haya cambios locales: `git status --short`.
2. Realice el respaldo según `docs/runbooks/postgresql-backup-restore.md`.
3. Descargue con `git pull --ff-only origin main`.
4. Lea las notas del Sprint y confirme el perfil Maven vigente.
5. Construya aplicación y migrador con el mismo perfil y etiquetas nuevas.
6. Actualice sólo `LOGIXONE_APP_IMAGE` y `LOGIXONE_MIGRATOR_IMAGE`.
7. Ejecute `docker compose config --quiet`.
8. Ejecute `up -d --wait` y revise el código de salida del migrador.
9. Compruebe `live`, `ready` y el login desde navegador.

```
cd "$HOME/proyectos/smart-erp"
git status --short
git pull --ff-only origin main
```

Regla de trazabilidad: no reconstruya una etiqueta existente para representar código distinto en un ambiente compartido. Use una etiqueta nueva y, para promover una versión aprobada, la referencia inmutable `repositorio@sha256:digest`.

Reversión

Volver a una imagen anterior no revierte migraciones de datos. El esquema evoluciona hacia adelante y las migraciones aplicadas son inmutables. Si la actualización falla:

1. detenga la aplicación;
2. conserve logs y código de salida del migrador;
3. no edite manualmente `flyway_schema_history`;
4. evalúe compatibilidad de la imagen anterior con el esquema ya migrado;
5. restaure desde respaldo sólo mediante el runbook aprobado y después de preservar la instancia fallida para diagnóstico.

13. Cambiar puertos o publicar detrás de un proxy

El valor seguro predeterminado es loopback. Si `8080` o `8180` están ocupados, no cambie sólo una línea: OIDC usa varias URL que deben permanecer coherentes.

Para usar, por ejemplo, aplicación `18080` y Keycloak `18180`, cambie juntos:

```
LOGIXONE_HTTP_PORT=18080
LOGIXONE_KEYCLOAK_PORT=18180
LOGIXONE_KEYCLOAK_PUBLIC_URL=http://keycloak.localhost:18180
LOGIXONE_OIDC_PROVIDER_URL=http://keycloak.localhost:18180/realms/logixone
LOGIXONE_OIDC_REDIRECT_URI=http://localhost:18080/logixone/*
LOGIXONE_OIDC_WEB_ORIGIN=http://localhost:18080
LOGIXONE_OIDC_POST_LOGOUT_REDIRECT_URI=http://localhost:18080/logixone/faces/app/index.xhtml
```

Después recree Keycloak y la aplicación, valide Compose y use la nueva URL. La publicación mediante proxy confiable, HTTPS y dominio real requiere configurar encabezados reenviados, orígenes y redirecciones como un cambio de despliegue revisado. No active `LOGIXONE_PROXY_ADDRESS_FORWARDING=true` detrás de un proxy que no sanee `X-Forwarded-*`.

14. Diagnóstico y recuperación

Orden de diagnóstico

1. `docker compose config --quiet` - valida configuración y rutas.
2. `docker compose ps -a` - identifica el primer servicio que no llegó al estado esperado.
3. logs de ese servicio - encuentra la causa, no sólo el efecto.
4. `health live` y `ready` - separa proceso vivo de dependencias disponibles.
5. espacio en disco y estado del motor - descarta una falla de plataforma.

Matriz de problemas frecuentes

Síntoma	Causa probable	Qué comprobar	Recuperación segura
<code>permission denied</code> al usar Docker	Usuario sin acceso al socket	<code>sudo docker version</code>	Use <code>sudo</code> ; no cambie permisos del socket a modo mundial.
Falta un archivo de secreto	Ruta o archivo vacío	<code>test -s .tools/secrets/...</code>	Vuelva a crear sólo el archivo faltante con <code>umask 077</code> . No cambie el Compose para escribir la contraseña en texto.
<code>migrator Exited (1)</code>	Migración, conexión o credencial fallida	Logs completos de <code>migrator</code> y salud de PostgreSQL	Deténgase. No arranque <code>app</code> a mano ni edite la historia Flyway.
<code>app</code> está <code>unhealthy</code>	DB, OIDC, plugins o despliegue no listos	Logs de <code>app</code> , <code>/health/live</code> , <code>/health/ready</code>	Corrija la dependencia indicada y recree <code>app</code> .
Bucle al iniciar sesión	URL/puerto OIDC incoherente	Variables de la sección 13, hora del sistema	Alinee todas las URL, recree Keycloak y <code>app</code> ; no desactive validaciones OIDC.
<code>keycloak.localhost</code> no responde desde otra PC	Sólo se tunelizó 8080	Túnel y puerto local 8180	Abra ambos forwards SSH, 8080 y 8180.
Puerto ya asignado	Otro servicio lo usa	<code>sudo ss -ltnp</code>	Detenga el servicio conocido o cambie todo el conjunto de URL.
Build sin espacio	Capas y caché consumen disco	<code>df -h</code> , <code>docker system df</code>	Identifique imágenes/caché no usadas. Nunca pode volúmenes del proyecto como limpieza genérica.
Los datos "desaparecieron"	Se usó otro nombre de proyecto Compose	<code>docker volume ls</code> , <code>COMPOSE_PROJECT_NAME</code>	Vuelva al nombre original. No cree datos nuevos encima antes de identificar el volumen correcto.

Comandos de diagnóstico

```
sudo systemctl status docker --no-pager
sudo docker info
sudo docker system df
sudo docker compose \
  --env-file infra/compose/compose.env.local \
  -f infra/compose/compose.yaml ps -a
sudo docker compose \
  --env-file infra/compose/compose.env.local \
  -f infra/compose/compose.yaml logs --no-color --tail=300
```

15. Lista de comprobación rápida

Primera instalación

- ☐ Ubuntu `x86_64` y recursos revisados.
- ☐ Docker Engine, Compose, Buildx y Git funcionan.
- ☐ Repositorio clonado desde GitLab y rama `main` limpia.
- ☐ Cuatro secretos creados con permisos `600`.
- ☐ `compose.env.local` creado sin contraseñas.
- ☐ Aplicación y migrador contruidos con el mismo perfil.
- ☐ `docker compose config --quiet` termina bien.
- ☐ PostgreSQL, Keycloak y app están saludables.
- ☐ Migrator termina con `Exited (0)`.
- ☐ `live` y `ready` responden HTTP 200.
- ☐ El navegador abre `http://localhost:8080/logixone/`.
- ☐ Se completa login y retorno a LogixOne.

Uso diario

```
cd "$HOME/proyectos/smart-erp"
sudo docker compose --env-file infra/compose/compose.env.local \
  -f infra/compose/compose.yaml up -d --wait --wait-timeout 240
curl -fsS http://127.0.0.1:8080/logixone/health/ready
```

Parada segura

```
sudo docker compose --env-file infra/compose/compose.env.local \
  -f infra/compose/compose.yaml down
```

16. Validación de fuentes y alcance de datos

Este manual se contrastó con los Dockerfiles, `infra/compose/compose.yaml`, la plantilla de entorno y los runbooks versionados del repositorio.

Con autorización del responsable, el 12 de agosto de 2026 también se inspeccionó en modo de solo lectura un entorno Compose aislado de desarrollo. No se consultaron filas de negocio y no se ejecutaron escrituras. La inspección confirmó:

- PostgreSQL 18.4;
- servicios `postgres`, `migrator`, `keycloak` y `app` en el patrón documentado;
- esquemas privados `core`, `plg_business_partners`, `plg_commercial_catalog`, `plg_inventory`, `plg_purchasing`, `plg_reference_data` y `plg_reference_plugin`;
- historias Flyway exitosas para kernel y plugins;
- funciones y triggers propios de los esquemas, sin usar actualización automática del esquema desde la aplicación;
- volúmenes separados para PostgreSQL y Keycloak.

Estos hallazgos sirven para verificar el recorrido operativo. No convierten el entorno inspeccionado en una instalación productiva ni sustituyen la validación independiente.

17. Seguridad, límites y soporte

Reglas de seguridad

- Nunca versione `.tools/secrets/` ni copie contraseñas a `compose.env.local`.
- Mantenga `LOGIXONE_HTTP_BIND=127.0.0.1` y `LOGIXONE_KEYCLOAK_BIND=127.0.0.1`.
- Para acceso remoto use túnel SSH o un despliegue TLS diseñado y revisado.
- No exponga PostgreSQL mediante `ports:`.
- No use `down --volumes` como comando de parada.
- No edite manualmente migraciones aplicadas ni `flyway_schema_history`.
- No ejecute `start-dev` de Keycloak como solución productiva.
- Antes de compartir logs, elimine secretos y datos personales innecesarios.

Accesibilidad del recorrido

Los comandos tienen resultado esperado y alternativa diagnóstica. En el navegador, use teclado y foco visible, no dependa únicamente del color de los estados y mantenga el zoom que necesite. Si una pantalla presenta desbordamiento horizontal o no permite completar la tarea con teclado, regístrelo como defecto funcional; no lo resuelva reduciendo permanentemente el zoom.

Al solicitar soporte incluya

1. fecha, commit (`git rev-parse HEAD`) y rama;
2. Ubuntu y arquitectura (`cat /etc/os-release` , `uname -m`);
3. versiones de Docker, Compose y Buildx;
4. salida de `docker compose ps -a` ;
5. endpoint que falla y código HTTP;
6. logs mínimos del servicio afectado, ya saneados;
7. si es instalación, actualización o reinicio;
8. acciones intentadas y su resultado.

No incluya archivos de `.tools/secrets` , tokens OIDC, contraseñas ni datos reales de clientes.

18. Referencias canónicas

- Compose del proyecto: `infra/compose/compose.yaml` .
- Configuración modelo: `infra/compose/compose.env.example` .
- Build: `docs/runbooks/docker-build.md` .
- Operación Compose: `docs/runbooks/compose.md` .
- Migraciones: `docs/runbooks/migrator.md` .
- Identidad: `docs/runbooks/keycloak-oidc.md` .
- Respaldo y restauración: `docs/runbooks/postgresql-backup-restore.md` .
- [Instalación oficial de Docker Engine en Ubuntu](#).
- [Posinstalación oficial de Docker en Linux](#).
- [Instalación oficial de Git](#).

Fuente canónica: este Markdown y los archivos versionados citados. El HTML y el PDF son artefactos derivados para consulta.